Predicting College Football

Win Probability

Engineered Play-by-Play Features · Dual-Model Architecture · Live Streamlit Application

Alvin Hampton

INFO 516 · Spring 2026

1.72M	8	24	0.924+
Play Events	FBS Seasons	Engineered Features	ROC-AUC

The Goal

Estimate the probability that a team wins at any point in the game — after every single play.

Use live game state

Score, time remaining, field position, down & distance, momentum — updated after every snap.

Produce calibrated probabilities

Not just picking a winner — estimating a probability that is trustworthy across the full 0–1 range.

Compare model architectures

Regularized Logistic Regression vs. XGBoost gradient boosted trees on 1.6M plays.

Deploy a live application

A Streamlit app that replays any game play-by-play and auto-detects the Play of the Game.

Evaluation Metrics

Win probability requires 4 metrics — no single number captures both classification correctness AND probability calibration.

Accuracy

> 80% is strong

Fraction of plays where the model correctly predicts the winner. Simple but ignores confidence — predicting 51% and 99% are treated identically.

ROC-AUC

> 0.85 strong, > 0.90 excellent

How consistently the model assigns higher probabilities to winning plays vs losing ones. 0.5 = random guessing. Our models: 0.924+

Brier Score

< 0.13 is strong (0.25 = baseline)

Mean squared error of predicted probabilities. Directly measures calibration — when the model says 70%, teams win ~70% of the time.

Log Loss

< 0.40 strong (0.69 = baseline)

Exponentially penalizes confident but wrong predictions. Ensures the model is honest about uncertainty, not just picking sides.

The Data: Scale & Scope

1,718,250	8 Seasons	362	10,000+
Individual Play Events	2017–2025 (excl. 2020)	Raw Feature Columns	FBS Games

Source:	collegefootballdata.com API — collected via cfbfastR (R package) for batch-download across all seasons.
Coverage:	All FBS programs: regular season, conference championships, bowl games, and postseason. Possession-level perspective.
Raw Dimensionality:	362 columns including deep text descriptions, team metadata, coordinate-level EPA/PPA metrics, and game state indicators.
Leaky Features Removed:	114 of 362 columns contained post-snap result information (yards gained, play EPA, outcome). Removed before any modeling.

Data Challenges: Preprocessing Pipeline

Clock Normalization — Temporal Discontinuity

⚠ **Issue:** Raw data resets the clock at halftime (1,800s → 0 per half). The model cannot distinguish a 2-minute drill before halftime vs. a 2-minute drill at game's end.

✓ **Fix:** Created adj_TimeSecsRem: a continuous countdown from 3,600 → 0. Periods 3 & 4 add remaining time to the 1,800s of the first half. Urgency is now mathematically consistent.

Red Zone Distance Anomaly — Field Position Corruption

⚠ **Issue:** Goal-to-go plays often listed distance = 10 yards even when the team was on the 2-yard line — a logical impossibility that corrupted field position modeling.

✓ **Fix:** Conditional cap: when Goal_To_Go flag is True, distance is forced to equal yards_to_goal. Simple, targeted, eliminates the noise.

Sparse Timeout Data — Imputation Strategy

⚠ **Issue:** Timeout values were frequently missing. Mean imputation is incorrect since timeouts reset to 3 each half and can only decrease — they are a monotonically non-increasing resource.

✓ **Fix:** Forward-fill grouped by game_id AND half. Propagates the last known timeout count forward and resets at the start of Q3. Logically consistent.

Data Challenges: Additional Noise & PySpark Pipeline

Duplicate Sequencing

~0.5% of games had skipped/duplicate play numbers. Fixed with secondary sort by period and TimeSecsRem.

Garbage Play Text

Hundreds of rows with 'BREAK' or 'null' as play text still had valid score/clock data. Re-classified or dropped if missing play_type.

Score Differential Spikes

Some rows recorded score changes before the touchdown play was logged. Recalculated pos_score_diff from start-of-play score.

Overtime Filtering

Period 5+ plays follow different rules (no clock, fixed yardage). Filtered entirely to prevent skewing time-remaining weights.

PySpark Distributed Pipeline

- Distributed chronological sort by game, period, clock, and play number
- Grouped window functions for rolling EPA and yards (lag-1 to prevent leakage)
- Forward-fill timeout imputation partitioned by game_id and half
- Skinny-down: drop 338 metadata columns → 24-feature dense matrix
- Convert to Pandas (.toPandas()) → scikit-learn modeling pipeline

The 24-Feature Architecture

Domain-informed engineering across 4 categories

Feature Set: Base Game State & Priors (11 Features)

- down, distance, yards_to_goal
- TimeSecsRem (adjusted — continuous)
- pos_score_diff
- pos_team_timeouts_rem_before
- def_pos_team_timeouts_rem_before
- is_home_team
- pos_team_elo
- def_pos_team_elo
- pos_team_spread

Why Possession Perspective?

Raw data ties features to the home team — forcing the model to learn team-specific patterns.

By reframing all features relative to the possession team, the model learns that being a 7-point favorite helps, not that Alabama usually wins. This generalizes across all matchups, including neutral-site and bowl games.

Vegas Spread & ELO as Priors

At kickoff, before any live game data exists, the spread and ELO ratings are the only meaningful signal available. They anchor the initial win probability to something more informative than a 50/50 coin flip — and their influence is progressively reduced by interaction terms as the game unfolds.

Feature Set: Momentum, Interactions & Polynomials (13 Features)

Drive Fatigue & Rolling Momentum (5)

- drive_yards_so_far — cumulative drive yardage
- drive_success_rate — % of successful plays
- drive_time_elapsed — drive fatigue proxy
- pos_rolling_EPA_last_5 — recent offensive efficiency
- pos_rolling_yards_last_5 — recent offensive production

All computed with lag(1) — the model never sees the outcome of the play it is currently predicting.

Polynomial Terms (3)

- poly_score_diff_sq — sign-preserved squared lead
- poly_TimeSecsRem_sq — squared time remaining
- poly_yards_to_goal_sq — squared field position

Captures non-linear football thresholds: a 14-pt lead is not 2× safer than 7 pts — it's exponentially safer.

Interaction Terms (5)

inter_score_x_time

Score diff × fraction of game elapsed. Being up 7 in Q4 is fundamentally different from being up 7 in Q1.

inter_spread_x_time

Spread × time remaining. Pre-game expectations decay toward zero as live game reality takes over.

inter_down_x_distance

Down × distance. 3rd & 10 is far more difficult than 1st & 10.

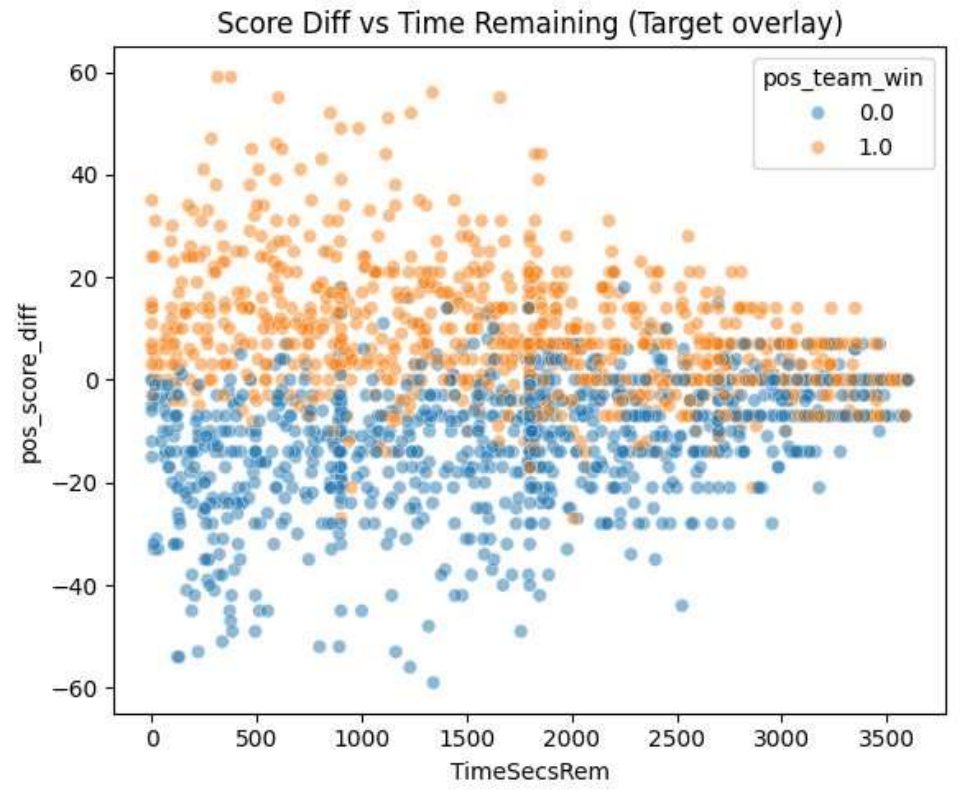
inter_elo_diff

Possession ELO minus Defense ELO. Net team strength from the ball-carrier's perspective.

inter_timeout_diff

Possession timeouts minus Defense timeouts. Late-game clock management leverage.

Why inter_score_x_time? — Visual Proof



What the scatter shows:

Left edge (time ≈ 0)

Orange (wins) and blue (losses) separate cleanly. A large positive score diff reliably predicts a win.

Right edge (time ≈ 3600)

Same score differentials produce heavily mixed outcomes — a big lead early means very little.

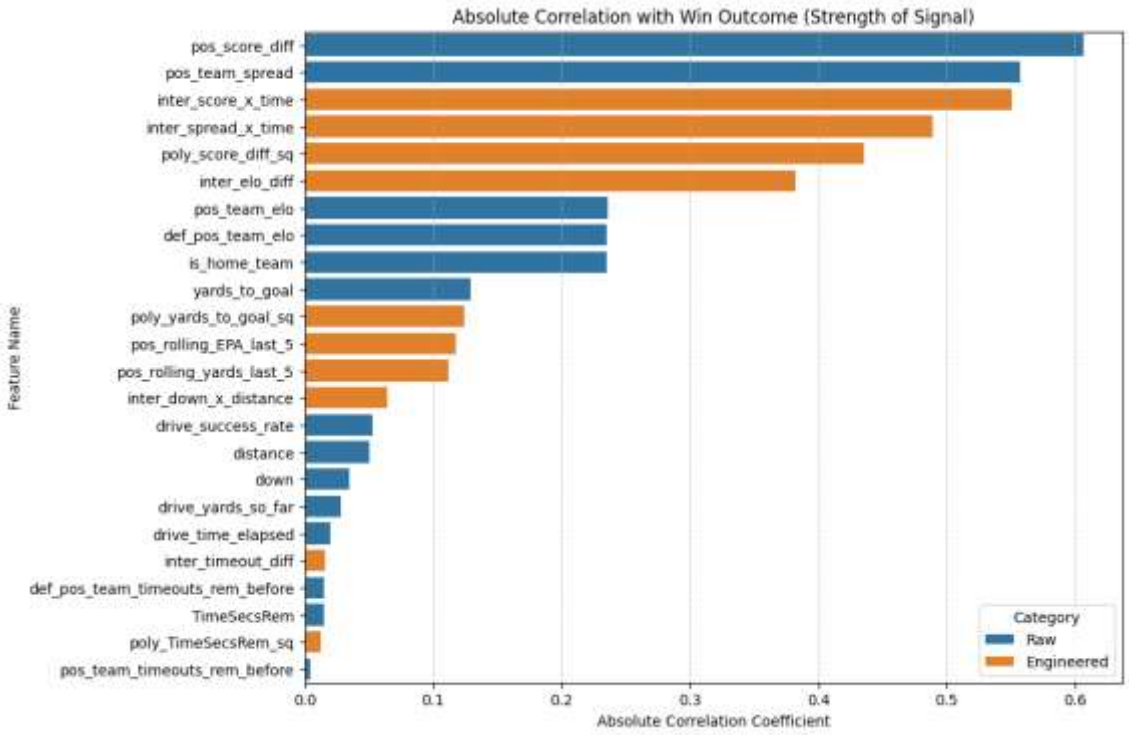
The Insight

The predictive value of a scoring lead is not fixed. It grows non-linearly as the clock expires.

The Feature

inter_score_x_time encodes this directly: score_diff × time_elapsed_fraction. At kickoff = 0. Late in Q4 = amplified.

Feature Signal Strength — Raw vs. Engineered



Key Finding

Engineered features (orange) are consistent toward the top of the correlation ranking despite being derived from raw features (blue). The model sees higher-quality signal from the constructed terms than from the raw inputs alone.

Efficiency Ratio

Engineered features are 54% of columns but carry ~60% of total predictive power.

1.38×

signal per column vs. raw features

Data Leakage Prevention — 3-Layer Framework

Layer 1 — Feature-Level: Post-Snap Removal

All 114 post-snap variables (yards gained, play EPA, play outcome) were excluded entirely. The model only sees information available to an observer before the ball is snapped.

114 Features Removed

Layer 2 — Temporal: Chronological Lagging

All rolling and cumulative momentum features were computed with a lag(1) window at the PySpark layer. Momentum statistics for play N reflect only plays 1 through N−1 — never the current play's result.

lag(1) enforced at Spark layer

Layer 3 — Split-Level: GroupShuffleSplit by Game ID

GroupShuffleSplit (not random split) was applied on game_id. All plays from a game are in training OR test — never split across both. A random split would let the model train on early plays and evaluate on later plays of the same game.

80/20 by game, not by play

Statistical Validation

Confirming feature significance with statsmodels Logit analysis

Logit Regression — Coefficient & Significance Analysis

```
                           Logit Regression Results
==============================================================================
Dep. Variable:            pos_team_win   No. Observations:              1590878
Model:                           Logit   Df Residuals:                  1590855
Method:                            MLE   Df Model:                           22
Date:                 Mon, 27 Apr 2026   Pseudo R-squ.:                  0.5010
Time:                         17:56:50   Log-Likelihood:            -5.5027e+05
converged:                        True   LL-Null:                   -1.1026e+06
Covariance Type:             nonrobust   LLR p-value:                     0.000
==========================================================================================
                             coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------------------
const                     -0.0331      0.003    -12.659      0.000      -0.038      -0.028
down                      -0.1030      0.006    -16.376      0.000      -0.115      -0.091
distance                  -0.0334      0.007     -4.767      0.000      -0.047      -0.020
yards_to_goal             -0.0191      0.011     -1.787      0.074      -0.040       0.002
TimeSecsRem               -0.0516      0.010     -5.329      0.000      -0.071      -0.033
pos_score_diff             1.1463      0.016     71.610      0.000       1.115       1.178
pos_team_timeouts_rem_before  0.0077  3.35e+04   2.29e-07      1.000  -6.57e+04   6.57e+04
def_pos_team_timeouts_rem_before  0.0069  3.32e+04   2.07e-07      1.000  -6.5e+04   6.5e+04
pos_team_spread           -0.6742      0.010    -69.330      0.000      -0.693      -0.655
pos_team_elo               0.0207        nan        nan        nan         nan         nan
def_pos_team_elo          -0.0143        nan        nan        nan         nan         nan
is_home_team               0.0641      0.003     25.062      0.000       0.059       0.069
drive_time_elapsed        -0.0268      0.002    -11.046      0.000      -0.032      -0.022
drive_yards_so_far        -0.0383      0.004     -8.774      0.000      -0.047      -0.030
drive_success_rate         0.0360      0.004     10.114      0.000       0.029       0.043
pos_rolling_EPA_last_5     0.0065      0.003      2.217      0.027       0.001       0.012
pos_rolling_yards_last_5   0.0126      0.003      4.403      0.000       0.007       0.018
inter_score_x_time         2.1484      0.017    129.325      0.000       2.116       2.181
inter_down_x_distance     -0.0042      0.008     -0.536      0.592      -0.020       0.011
inter_elo_diff             0.0284        nan        nan        nan         nan         nan
inter_timeout_diff         0.0007  4.09e+04   1.75e-08      1.000  -8.02e+04   8.02e+04
inter_spread_x_time       -0.7047      0.008    -85.363      0.000      -0.721      -0.688
poly_TimeSecsRem_sq       -0.0113      0.009     -1.232      0.218      -0.029       0.007
poly_yards_to_goal_sq     -0.1447      0.010    -14.349      0.000      -0.165      -0.125
poly_score_diff_sq        -0.3505      0.030    -11.570      0.000      -0.410      -0.291
==========================================================================================
```

Pseudo R^2 = 0.501

The 24 features explain 50% of the maximum possible log-likelihood improvement. Strong result for play-by-play prediction.

inter_score_x_time z = 129.3 p = 0.000

The single most statistically significant predictor in the entire model — confirming the urgency interaction is real, not noise.

pos_score_diff, spreads, poly terms

All $p < 0.001$. Core game state and engineered features are highly significant across the board.

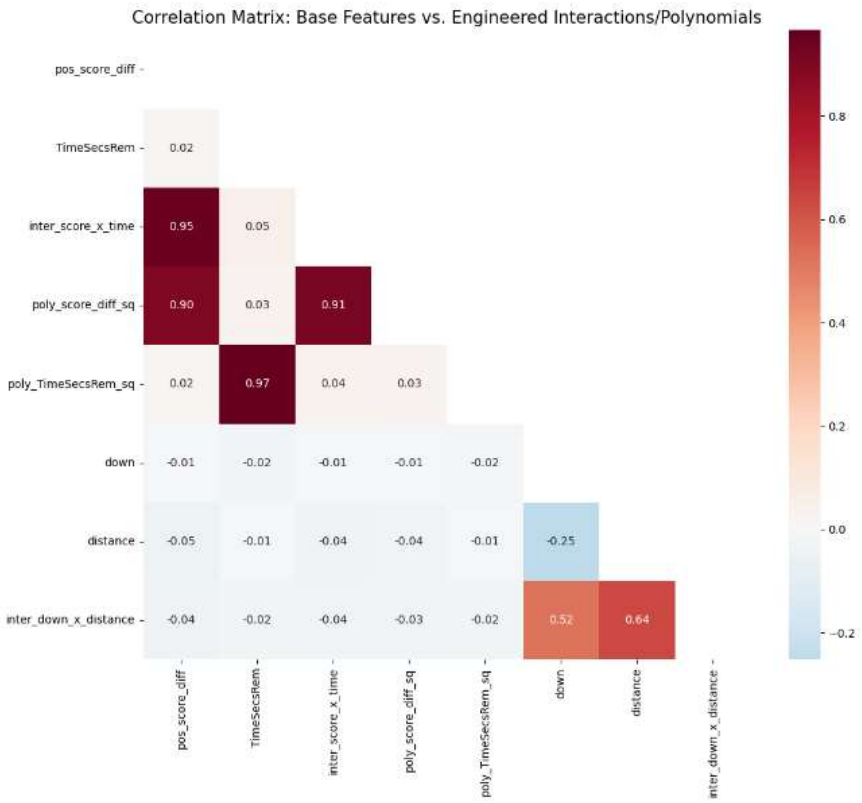
NaN p-values: ELO features

inter_elo_diff = pos_elo − def_elo exactly. Perfect multicollinearity → singular matrix. L2 regularization resolves this in production.

Timeout features p = 1.000, std err ≈ 33,000

Classic multicollinearity explosion between individual timeout counts and inter_timeout_diff. Stabilized by L2 penalty in production models.

Multicollinearity — Detected, Explained & Resolved



What the heatmap shows

inter_score_x_time correlates 0.95 with pos_score_diff. poly_TimeSecsRem_sq correlates 0.97 with TimeSecsRem. These are built-in by design — interaction terms are functions of their base variables.

Why it doesn't hurt performance

Multicollinearity inflates standard errors (see NaN p-values) but doesn't degrade predictive accuracy. The related features collectively provide richer signal than either could alone.

Logistic Regression fix: L2 regularization (C=0.1)

L2 penalty shrinks all coefficients proportionally, distributing predictive weight across correlated features rather than assigning explosive weights that produce erratic probability jumps.

XGBoost fix: Natural tree splitting

Tree models select the most informative feature at each node. Redundant correlated features are simply ignored at splits where a better alternative exists so no special handling required.

Dual-Model Architecture

Logistic Regression · XGBoost

The Linear Model — Regularized Logistic Regression

Algorithm

Logistic Regression with L2 (Ridge) penalty. Each feature has a coefficient indicating both magnitude and direction of its influence on win probability.

Solver

SAGA — specifically designed for datasets exceeding 1M rows. Efficient stochastic gradient convergence without memory limits.

Regularization (C = 0.1)

Strong L2 penalty distributes weight across correlated features, preventing explosive coefficients that would cause erratic probability jumps in the app.

Hyperparameter Tuning

GridSearchCV with 3-fold cross-validation on the grouped training set. Best C identified then model retrained on full 8-season dataset.

Interaction-Heavy Design

Logistic Regression is linear — it cannot discover non-linear relationships on its own. I pre-built them: inter_score_x_time, poly_score_diff_sq, etc.

Coefficient Transparency

Every feature has an interpretable weight. A 1-unit increase in pos_score_diff increases log-odds of winning by a fixed, measurable amount.

The Tree Model — XGBoost Gradient Boosted Classifier

Algorithm

Gradient Boosted Decision Trees (GBDT). Builds 500 shallow trees sequentially, each correcting the errors of the previous ensemble.

tree_method = 'hist'

Histogram-based splitting binarizes continuous features into buckets thus enabling fast training on 1.6M rows with reduced memory overhead.

max_depth = 3 (deliberate)

Shallow trees prevent overfitting to game-specific noise and avoid the steppy, jagged probability curves that deep trees can produce in live applications

learning_rate = 0.01

Conservative updates: each tree contributes a small, cautious correction to the ensemble rather than aggressively fitting residuals. Improves generalization.

Hyperparameter Tuning

RandomizedSearchCV over 500 estimators, learning rate [0.01, 0.05, 0.1], max_depth [3, 5, 7], subsampling 0.8.

Non-Linear Discovery

Unlike LogReg, XGBoost discovers thresholds automatically: '3rd & Short vs 3rd & Long', Red Zone leverage, and two-minute-drill urgency are all found through splits.

Training Process

GroupShuffleSplit

80 / 20 by game_id

Split all 1.6M plays into training and test by whole game. Ensures evaluation is on games the model has never seen in any form.

Cross-Validation

GridSearchCV / RandomizedSearchCV

LogReg: GridSearchCV over C values. XGBoost: RandomizedSearchCV over depth, learning rate, and subsampling. Both: 3-fold CV on training set only.

Best Parameters

LogReg: C = 0.1 · XGBoost: depth=3, lr=0.01, n=500

Optimal hyperparameters selected based on validation log loss across all CV folds.

Full Retrain

Train + Test recombined

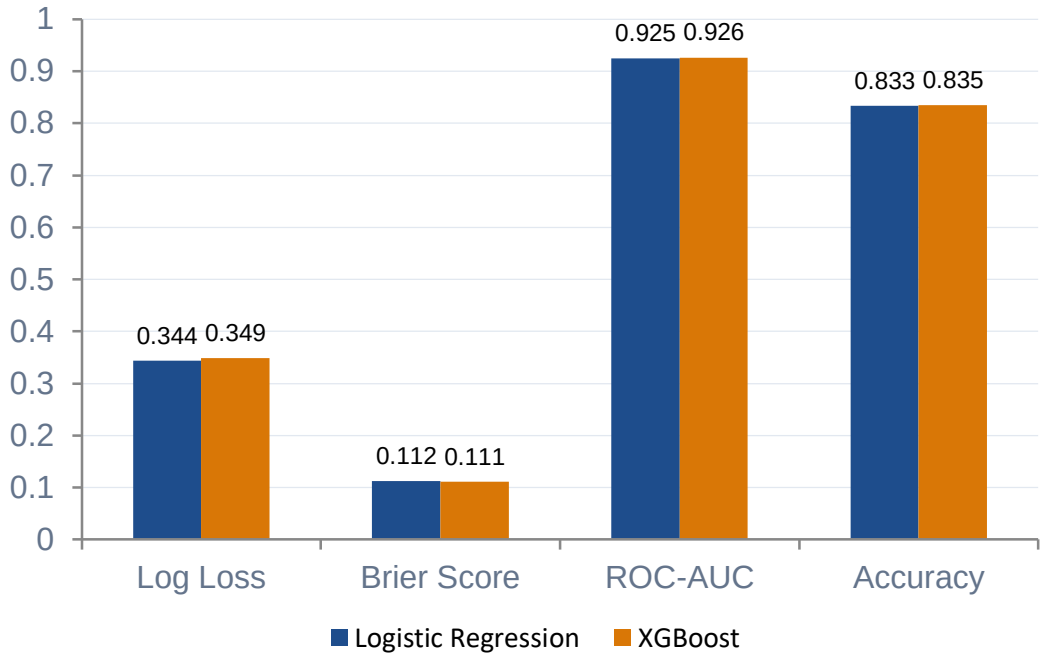
After CV, training and test sets are merged back together. Both models are retrained on the complete 8-season dataset so every play contributes to the production model.

Export & Deploy

.pkl files → Streamlit app

Models and StandardScaler serialized with joblib. Loaded into Streamlit at startup via @st.cache_resource and cached for session-length inference.

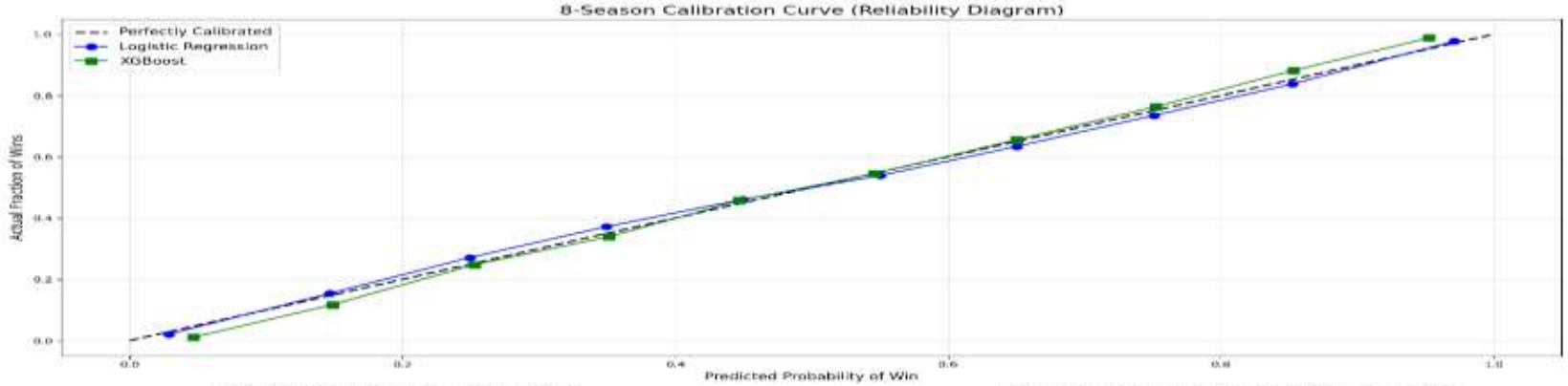
Model Results — 8-Season Test Set (~318,000 held-out plays)

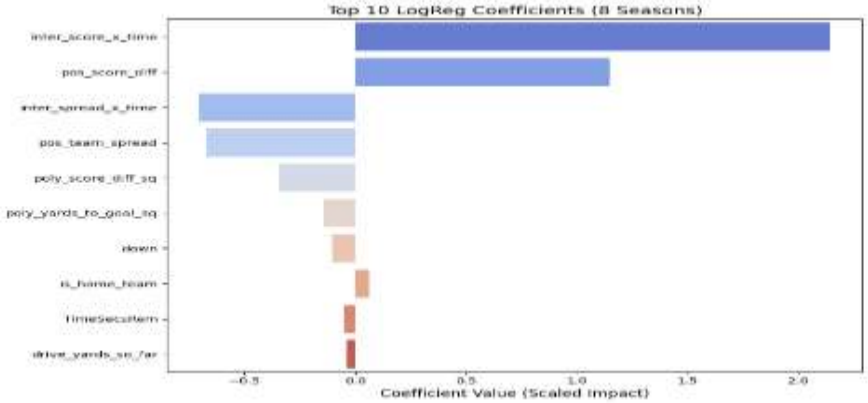


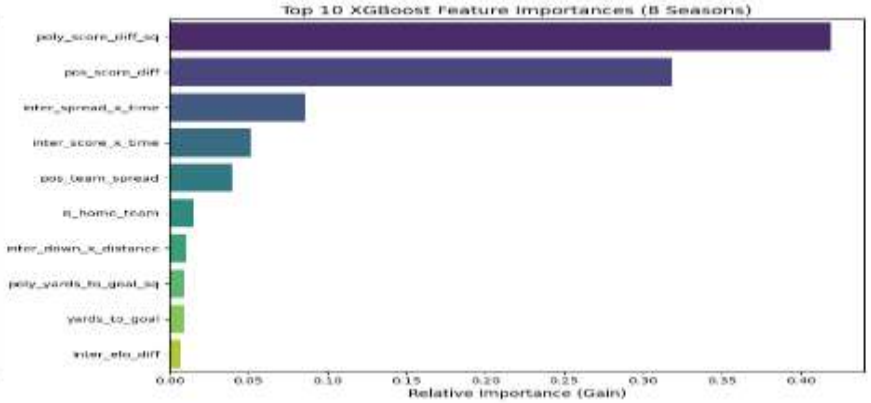
Metric	LogReg	XGBoost
Log Loss	0.3437	0.3488
Brier Score	0.1121	0.1113
ROC-AUC	0.9245	0.9263
Accuracy	83.34%	83.51%

Both models converge to the same performance — a direct consequence of the feature engineering making the problem effectively linear.

Calibration & Feature Importance — Production Models







Top: Both models track the perfectly calibrated baseline closely across the full 0–1 range. | Bottom-left: LogReg — inter_score_x_time coefficient dominates (2.14×). | Bottom-right: XGBoost — poly_score_diff_sq accounts for 41.9% of total gain.

The Takeaway — Why Linear ≈ XGBoost

Feature Engineering Linearized the Problem

We gave LogReg the non-linear signals it cannot discover: inter_score_x_time, poly_score_diff_sq, inter_spread_x_time. A linear model given the right compound inputs doesn't need to discover the physics — it just learns the weights.

02

Football Has a Strong Linear Core

The core drivers — score, time, field position — follow a largely predictable progression. XGBoost handles edge cases (4th-and-1 from the 1, rare upsets) better, but those situations are rare enough that the aggregate impact is marginal.

Diminishing Returns at AUC > 0.92

Once 0.924 AUC is captured, you've extracted nearly all the predictable information in the dataset. Moving from 92.4% to 93% can require 10× the model complexity (e.g., LSTMs) for a <1% gain. Feature quality matters more than model complexity.

Streamlit Application

Live Win Probability Tracker — Deployed Production Engine

Application Architecture — 3 Core Decisions

Perspective-Flip Logic — The Continuous Graph Secret

The model predicts possession-team win probability. On a turnover, the raw output would jump (80% → 20%) creating visual discontinuities. The app anchors all output to the home team: if pos_team == home_team: display WP, else: display (1 − WP). Turnovers appear as smooth dips or spikes — matching ESPN-style broadcast charts.

No Chart Jumps

Win Probability Swing — Automatic Play of the Game

After computing WP for every play, the app calculates the delta between consecutive plays from the winning team's perspective. The play with the largest positive swing is flagged as the Play of the Game — automatically surfacing the game's pivotal moment without any manual analysis.

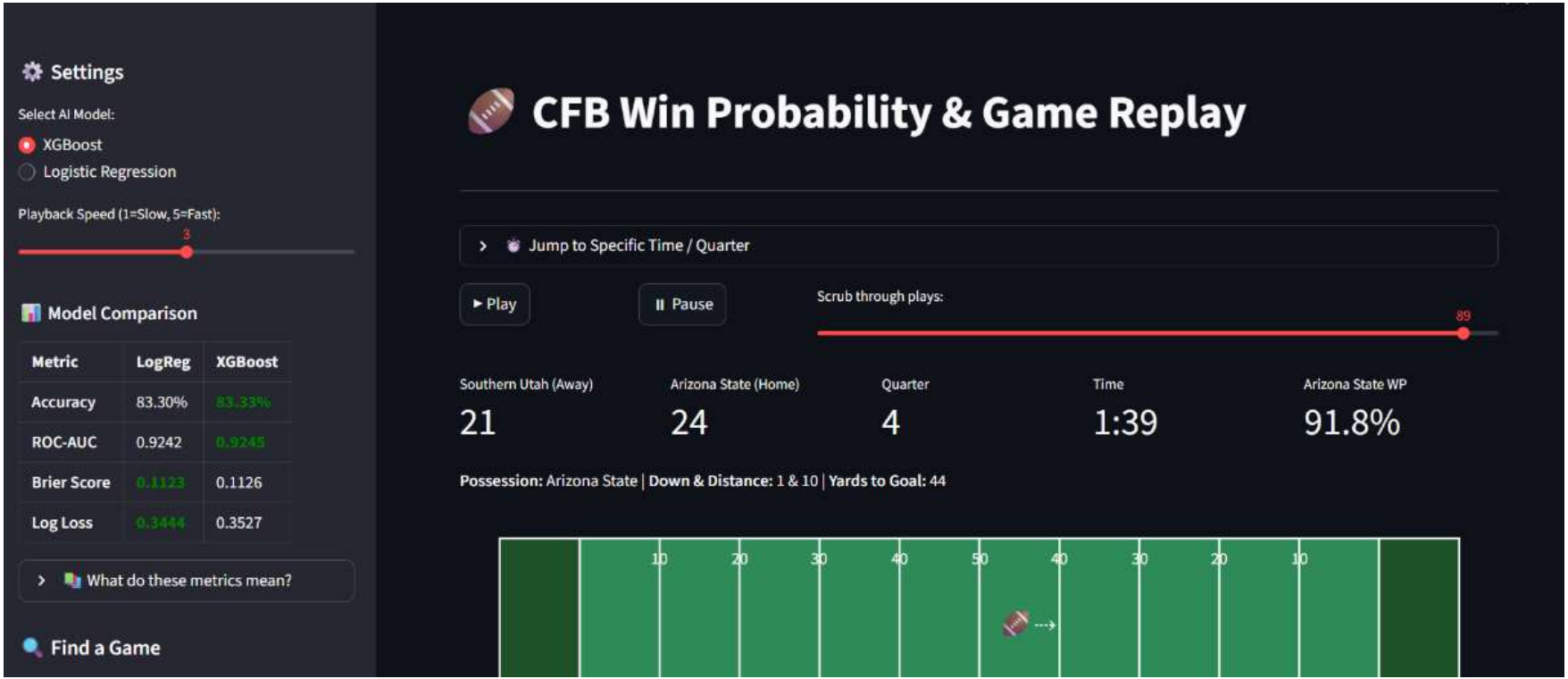
Auto-Detected

@st.cache_resource — Optimized Inference

Both models and the fitted StandardScaler are loaded into memory at startup and cached for the full session. No per-request disk I/O. The app processes all ~200 plays of a typical game and renders the complete WP chart in under 1.5 seconds.

< 1.5 sec per game

Live Application — CFB Win Probability & Game Replay



Sidebar: model selection + live metric comparison · Main panel: live score, possession, down/distance, field position · Win probability: 91.8% with 1:39 remaining (Arizona State vs Southern Utah, Q4)

Key Results

0.9245–0.9263	83.3–83.5%	0.501	1.38×	< 1.5 sec
ROC-AUC	**Accuracy**	**Pseudo R^2**	**Feature Efficiency**	**Full Game Inference**
Both models	*318K test plays*	*Statsmodels Logit*	*Engineered vs. raw*	*Streamlit app*

Core Conclusion

Domain-informed feature engineering closed the performance gap between a linear model and a 500-tree gradient boosted ensemble. The engineered features are statistically significant (z = 129.3 for inter_score_x_time), non-redundant by design, and deliver 1.38× more signal per column than raw inputs. Feature quality beats model complexity.